

בעיות תכנון בתור SAT

אלגוריתם DPLL משפר שני דברים באלגוריתם של SAT:

- אדום - מפשט את Φ .
- כוחל - בוחר משתנה חדש וbranch חדש
- DPLL מתמקד באדום באמצעות שני חוקים:
 - Unit propagation - מסתכל על clause שמכילים רק ליטרל אחד (למשל (x) או $(\neg y)$), ולפי זה מסיקים מה הערך של המשתנה הזה חייב להיות.
 - Pure literal elimination - מזהה משתנה p שהוא טהור - או שהוא מופיע תמיד p או שהוא מופיע תמיד $\neg p$. זה אומר שאפשר לקבוע אותו לו תמיד ערך אמת או תמיד ערך שקר בלי קשר לשאר המשתנים.

איך ממירים בעיית תכנון לבעיית SAT? בעיית מסלול ב SAT, באופן כללי, היא PSPACE-complete. אבל הבעיה של לבדוק אם קיים מסלול באורך מסויים היא NP-complete. לכן אפשר להגדיל כל פעם את אורך המסלול $b \leftarrow b + 1$, לבנות CNF שמייצג אם קיים מסלול בן b צעדים, להריץ DPLL, ואם הוא מחזיר פתרון - המסלול שלנו מקודד בתוך הפתרון הזה.

איך מקודדים בעיית STRIPS בתור CNF?

- הבעיה $\Pi = (P, A, I, G)$ עם $0 \leq t \leq b$ time steps.
- משתני החלטה: לכל אטום בכל זמן ניצור משתנה:
 - לכל פרדיקט $p \in P$ ניצור p_t עבור $0 \leq t \leq b$
 - לכל פעולה $a \in A$ ניצור a_t עבור $0 \leq t \leq b - 1$
- המצב ההתחלתי p_0 :
 - אם $p \in I$ נוסף את clause (p_0) .
 - אם $p \notin I$ נוסף את clause $(\neg p_0)$.
- המטרה p_b : לכל $p \in G$ נוסף את clause (p_b) .
- preconditions: אפשר להפעיל פעולה a רק אם התנאי p מתקיים - כלומר $a_t \rightarrow p_t$, ולכן נוסף את clause $(\neg a_t \vee p_t)$
- אפקטים של הוספה - $a_t \rightarrow p_{t+1}$, ולכן נוסף את clause $(\neg a_t \vee p_{t+1})$.
- אפקטים של הסרה - $a_t \rightarrow \neg p_{t+1}$, ולכן נוסף את clause $(\neg a_t \vee \neg p_{t+1})$.
- positive frame - אם פעולה a קוראת, כל התנאים p שהיא לא נוגעת בהם נשארים דלוקים: $(a_t \wedge p_t) \rightarrow p_{t+1}$. לכן נוסף $(\neg a_t \vee \neg p_t \vee p_{t+1})$.
- באותו אופן negative frame להשאיר תנאים שהיא לא נוגעת בהם כבויים.
- רק פעולה אחת יכולה להיות מופעלת בכל זמן - נצטרך להוסיף clause גם בשביל זה. לכל זוג פעולות a, a' נוסף $(\neg a_t \vee \neg a'_t)$.
- חייבים לפחות פעולה אחת - ולכן צריך להוסיף clause גדול $\bigvee_{a \in A} a_t$.

Parallel encoding

מתי איכפת לי מהסדר בין 2 פעולות בSTRIPS? כאשר delete effect של אחת נמצא בpreconditions של השני. אם אין חפיפה כזאת, אז אפשר להחליף בין הסדר של הפעולות. לכן נחליף את התנאי "רק פעולה אחת" בתנאי mutex constraints שקובע מה אפשר להפעיל ביחד ומה אי אפשר. כלומר נוסיף פסוקיות $(\neg a_t \vee a'_t)$ אם $a \not\leq a'$ - כלומר רק אם הם לא בלתי תלויים.

Branch and Bound

Branch and Bound זה אלגוריתם לבעיות אופטימיזציה שמתייחס להיסטוריה:

- אם מצאתי משהו, זה "הכי טוב עד עכשיו"
 - אם מצאתי משהו יותר טוב - מעדכנים
 - אם מצאתי משהו פחות טוב - אפשר לעשות pruning
 - בניגוד ל- A^* , שם אי אפשר למחוק כי עוד לא הגענו למטרה.
- בעצם קודם מחפשים פתרון, ואז מחפשים איך לשפר אותו.
- $OPEN$ שלנו יהיה תור עדיפויות מסודר לפי $B(n)$ - bound שנותן שיערוך לפתרון החלקי.
- הרעיון: להגדיר מה צומת מייצג (פתרון חלקי)
 - להגדיר את ה- $B(n)$ bound (שיערוך אופטימי להשלמת n לפתרון)
 - לקטום לפי ה- $bestn$ הנוכחי.

BnB נאיבי

קודקוד חיפוש הוא $n = \langle A, V, C \rangle$ כאשר:

A	השמה נוכחית
V	משתנים שעדיין לא שמתו להם השמה
C	אילוצים - גם המטרה וגם פונקציות המחיר

ה- $Bestn$ הוא קודקוד שסיים את ההשמה וה- $B(n)$ שלו הכי טוב עד כה. מאתחלים $Best \leftarrow \infty$, ובכל פעם שמגיעים לפתרון (כלומר $V(n) = \emptyset$ - לא נשארו יותר השמות) מעדכנים $Best \leftarrow \min(Best, cost(A(n)))$ ¹.

כאשר מפתחים, שומרים ב- $OPEN$ רק קודקודים n' עבורם $B(n') < Best$. למה זה נאיבי? כי זה עושה המון השמות. זה מתפוצץ הרבה לפני שמגיעים להשמה מלאה (שמאפשרת לנו לעשות pruning) - בשלב הראשון יש לנו $|V| \cdot d$ ילדים (כאשר d הוא גודל Domain), וכל ילד כזה מייצר לנו בשלב השני $(|V| - 1) \cdot d$ ילדים - וכן הלאה.

Deferred Computation

בשביל להימנע מהפיצוץ האקספוננציאלי, נשנה את מבנה הקודקוד ל- $n = \langle A, V, C, U(v) \rangle$ כאשר $U(v)$ קובע לכל משתנה v מה הערכים שעוד לא ניסינו. זה מאפשר לנו לפתח רק מספר מצומצם של ילדים כל פעם. כאשר באים לפתח צומת n (כלומר $(n) \neg \text{Complete?}$):

- בוחרים משתנה. בניגוד לאלגוריתם הנאיבי, שבו עוברים בלולאה על כל המשתנים, כאן בוחרים רק אחד ולכן אפשר לבחור אותו בצורה חכמה.

¹ $cost(n)$ זה המחיר האמיתי. $B(n)$ זה יוריסטיקה.

בינה מלאכותית
89-5570-10

מקליד: עידן אריה
מתרגל: דוד יצחקי
תאריך: 2026-06-25

-
- אם $v \in Vars(n)$, אז זו הפעם הראשונה שיש לו השמה - ולכן מסירים אותו מ- $Vars(n)$. בכל מקרה בוחרים לו הצבה.
 - בוחרים k ערכים ל- v שעדיין לא ניסינו. גם כאן, אפשר לבחור בצורה חכמה שכן אנחנו לא חייבים לעבור על הכל.
 - מכיוון שלא פיתחנו עדיין את כל הילדים האפשריים של n , צריך להחזיר אותו חזרה ל- $OPEN$ (אחרי שמעדכנים את ה- $U(v)$ שלו)

בעיית תכנון בתור CSP

כמו שיכולנו למדל את בעיית התכנון בתור SAT - אפשר גם למדל בתור CSP.

- הרעיון: המשתנים $x^1, \dots, x^n$ מדומיינים $D_1, \dots, D_n$.
- יוצרים עותק $x_t^i \in D$ לכל $t = 0, \dots, k$ time step ולכל משתנה $i = 1, \dots, n$.
- זה אומר שכל מצב הוא ווקטור $s_t = (x_t^1, \dots, x_t^n)$.
- לכל זמן $t = 0, \dots, k-1$ מוספים משתנה פעולה $A_t \in A$.
- A_t - היא הפעולה שמבצעים בזמן t .
- האילוצים יחברו את (s_t, A_t, s_{t+1}) .
- אם יש פעולה a שדורשת $x^i = v$, אז בכל זמן t צריך $A_t = a$ יגרור $x_t^i = v$ - כלומר (בייצוג CSP):

$$(A_t \neq a) \vee (x_t^i = v)$$

- אם יש פעולה a שגוררת $x^j = u$, אז בכל זמן t צריך $A_t = a$ יגרור $x_{t+1}^j = u$ - כלומר (בייצוג CSP):

$$(A_t \neq a) \vee (x_{t+1}^j = u)$$

- negative frame ו positive frame כמו ב SAT.

גם כאן, כמו ב SAT, מנסים עם $k=1, k=2, k=3$ וכן הלאה עד שמוצאים משהו. וגם כאן זה לא מתחשב במחירים. וגם כאן אפשר לעשות parallel.